Optimization

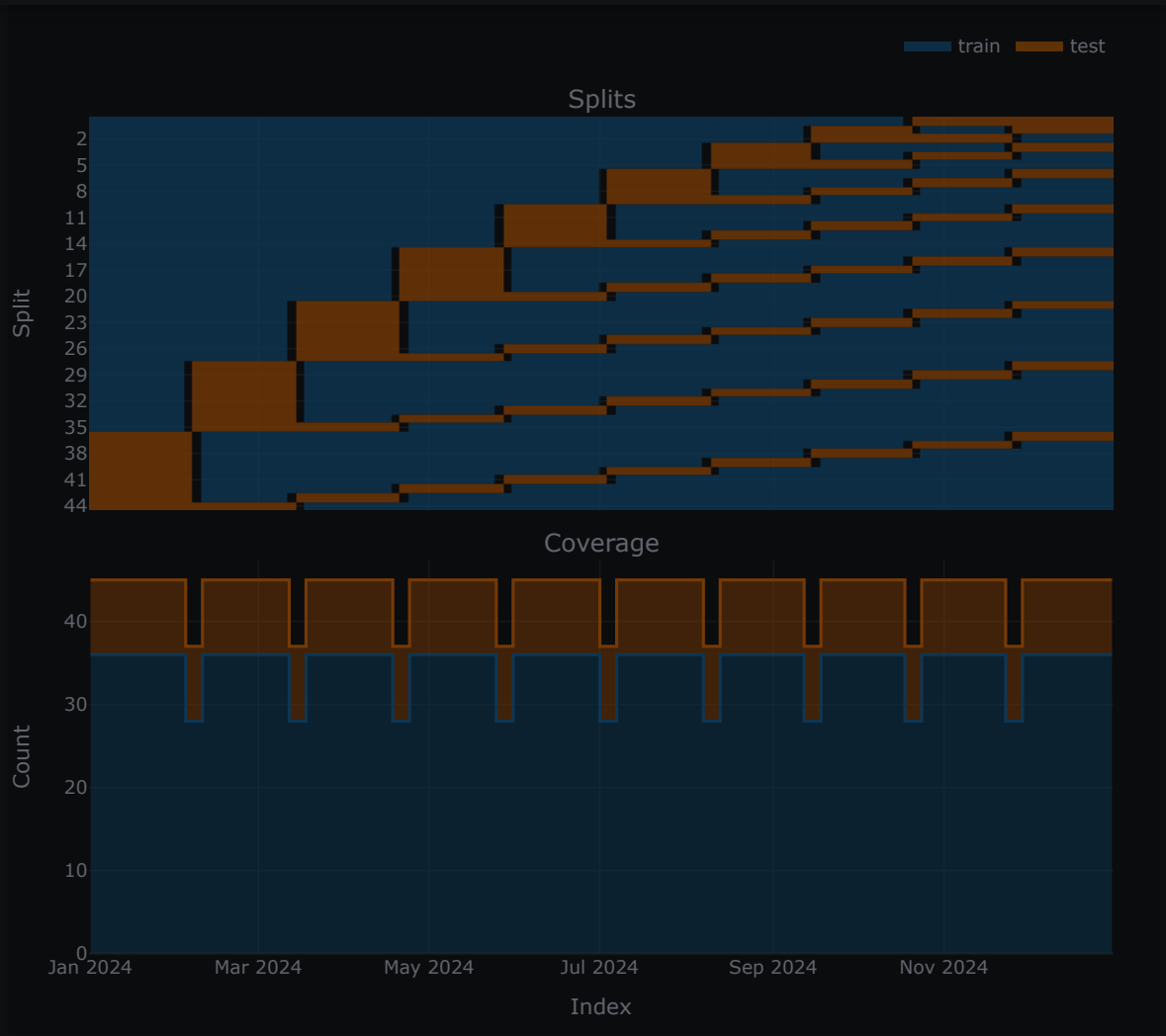
Purged CV

new in v2024.4.1

- Added support for walk-forward cross-validation (CV) with purging, as well as combinatorial CV with purging and embargoing, based on Marcos Lopez de Prado's Advances in Financial Machine Learning.

Create and plot a combinatorial splitter with purging and embargoing

```
>>> splitter = vbt.Splitter.from_purged_kfold(
...     vbt.date_range("2024", "2025"),
...     n_folds=10,
...     n_test_folds=2,
...     purge_td="3 days",
...     embargo_td="3 days"
... )
>>> splitter.plots().show()
```



Paramables

new in v2024.4.1

- Each analyzable VBT object (such as data, indicator, or portfolio) can now be split into items, which are multiple objects of the same type, each containing only one column or group. This makes it possible to use VBT objects as standalone parameters and process only a subset of information at a

```
>>> data = vbt.YFData.pull(["BTC-USD", "ETH-USD"])
>>> sma = data.run("talib:sma", timeperiod=range(20, 50, 2))   (2)
>>> fast_sma = sma.rename_levels({"sma_timeperiod": "fast"})   (3)
>>> slow_sma = sma.rename_levels({"sma_timeperiod": "slow"})
>>> entries, exits = get_signals(
...     vbt.Param(fast_sma, condition="__fast__ < __slow__"),   (4)
...     vbt.Param(slow_sma)
... )
>>> entries.columns
MultiIndex([(20, 22, 'BTC-USD'),
            (20, 22, 'ETH-USD'),
            (20, 24, 'BTC-USD'),
            (20, 24, 'ETH-USD'),
            (20, 26, 'BTC-USD'),
            (20, 26, 'ETH-USD'),
            ...
            (44, 46, 'BTC-USD'),
            (44, 46, 'ETH-USD'),
            (44, 48, 'BTC-USD'),
            (44, 48, 'ETH-USD'),
            (46, 48, 'BTC-USD'),
            (46, 48, 'ETH-USD')],
           names=['fast', 'slow', 'symbol'], length=210)
```

(1) Regular function that takes two indicators and returns signals.

(2) Run an SMA indicator once on all time periods.

(3) Copy the indicator and rename the parameter level to get a distinct indicator instance.

(4) Pass both indicator instances as parameters. This splits each instance into smaller instances with only one column. Also, remove all columns where the fast window is greater than or equal to the slow window.

Lazy parameter grids

new in v2023.12.23

- The parameterized decorator no longer needs to materialize parameter grids if you are only interested in a subset of all parameter combinations. This change enables the generation of random parameter combinations almost instantly, no matter how large the total number of possible combinations is.

```
...         tp_stop=tp_stop,
...     ).total_return

>>> n = np.arange(10, 100)
>>> sl_stop = np.arange(1, 1000) / 1000
>>> tsl_stop = np.arange(1, 1000) / 1000
>>> tp_stop = np.arange(1, 1000) / 1000
>>> len(n) * len(sl_stop) * len(tsl_stop) * len(tp_stop)
89730269910

>>> test_combination(
...     vbt.YFData.pull("BTC-USD"),
...     n=vbt.Param(n),
...     sl_stop=vbt.Param(sl_stop),
...     tsl_stop=vbt.Param(tsl_stop),
...     tp_stop=vbt.Param(tp_stop),
...     _random_subset=10
... )
n   sl_stop  tsl_stop  tp_stop
34  0.188    0.916     0.749       6.869901
44  0.176    0.734     0.550       6.186478
50  0.421    0.245     0.253       0.540188
51  0.033    0.951     0.344       6.514647
    0.915    0.461     0.322       2.915987
73  0.057    0.690     0.008      -0.204080
74  0.368    0.360     0.935      14.207262
76  0.771    0.342     0.187      -0.278499
83  0.796    0.788     0.730       6.450076
96  0.873    0.429     0.815      18.670965
dtype: float64
```

Mono-chunks

new in 1.13.0

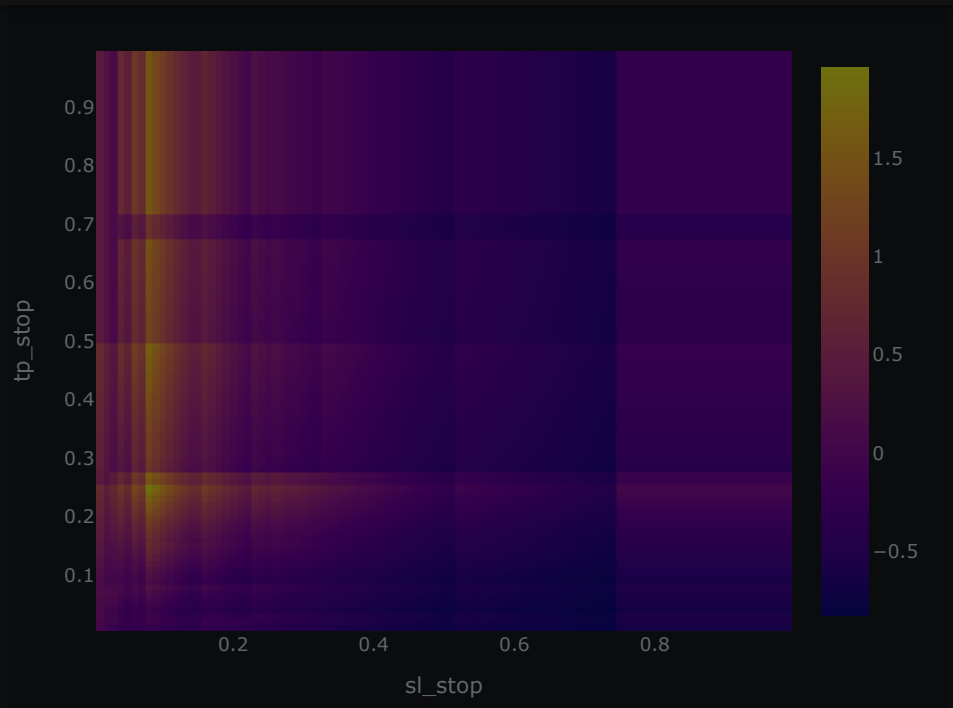
The parameterized decorator now supports splitting parameter combinations into "mono-chunks," merging the parameter values within each chunk into a single value, and running the entire chunk with a single function call. This means you are no longer limited to processing only one parameter combination at a time 🌪️ Keep in mind that your function must be adapted to handle multiple parameter values, and you should modify the merging function as needed.

Test 100 combinations of SL and TP values per thread

```
...     sim_out = vbt.pf_nb.from_signals_nb(
...         target_shape=(close.shape[0], sl_stop.shape[1]),
...         group_lens=np.full(sl_stop.shape[1], 1),
...         close=close,
...         long_entries=entries,
...         short_entries=exits,
...         sl_stop=sl_stop,
...         tp_stop=tp_stop,
...         save_returns=True
...     )
...     return vbt.ret_nb.total_return_nb(sim_out.in_outputs.returns)

>>> data = vbt.YFData.pull("BTC-USD", start="2020")  5
>>> entries, exits = data.run("randnx", n=10, hide_params=True, unpack=True)  6
>>> sharpe_ratios = test_stops_nb(
...     vbt.to_2d_array(data.close),
...     vbt.to_2d_array(entries),
...     vbt.to_2d_array(exits),
...     sl_stop=vbt.Param(np.arange(0.01, 1.0, 0.01), mono_merge_func=np.column_stack),
 7
...     tp_stop=vbt.Param(np.arange(0.01, 1.0, 0.01), mono_merge_func=np.column_stack)
... )
>>> sharpe_ratios.vbt.heatmap().show()
```

1. 100 values are combined into one array, forming a single mono-chunk.
2. Execute N mono-chunks in parallel, where N is the number of cores.
3. Use multithreading.
4. Execute one mono-chunk to compile the function before distributing other chunks.
5. The function above operates with only one symbol.
6. Pick 10 entries and exits randomly.
7. For each mono-chunk, stack all values into a two-dimensional array.



CV decorator

new in 1.8.1

Most cross-validation tasks involve testing a grid of parameter combinations on the training data, selecting the best parameter combination, and validating it on the test data. This process must be repeated for each split. The cross-validation decorator combines the parameterized and split decorators to automate this task.

Tutorial

Learn more in the Cross-validation tutorial.

Cross-validate a SMA crossover using random search

```
>>> @vbt.cv_split(
```

```
...     exits = fast_sma.real_crossed_below(slow_sma)
...     pf = vbt.PF.from_signals(data, entries, exits, direction="both")
...     return pf.deep_getattr(metric)

>>> sma_crossover_cv(
...     vbt.YFData.pull("BTC-USD", start="4 years ago"),
...     vbt.Param(np.arange(20, 50), condition="x < slow_period"),
...     vbt.Param(np.arange(20, 50)),
...     "trades.expectancy"
... )
```

Split 7/7

```
split  set    fast_period  slow_period
0      train  20           25              8.015725
       test   20           23              0.573465
1      train  40           48             -4.356317
       test   39           40              5.666271
2      train  24           45             18.253340
       test   22           36            111.202831
3      train  20           31             54.626024
       test   20           25             -1.596945
4      train  25           48             41.328588
       test   25           30              6.620254
5      train  26           32              7.178085
       test   24           29              4.087456
6      train  22           23             -0.581255
       test   22           31             -2.494519
dtype: float64
```

Split decorator

new in 1.8.1

Normally, to run a function on each split, you need to build a splitter specifically targeted at the input data provided to the function. This means that each time the input data changes, you must recreate the splitter. The split decorator automates this process by wrapping the function, giving it access to all arguments so it can make splitting decisions as needed. Essentially, it can "infect" any Python function with splitting functionality

```
...     takeable_args=["data"],
...     merge_func="concat"
... )
... def get_quarter_return(data):
...     return data.returns.vbt.returns.total()

>>> data = vbt.YFData.pull("BTC-USD")
>>> get_quarter_return(data.loc["2021"])
Date
2021Q1    1.005805
2021Q2   -0.407050
2021Q3    0.304383
2021Q4   -0.037627
Freq: Q-DEC, dtype: float64

>>> get_quarter_return(data.loc["2022"])
Date
2022Q1   -0.045047
2022Q2   -0.572515
2022Q3    0.008429
2022Q4   -0.143154
Freq: Q-DEC, dtype: float64
```

Conditional parameters

new in 1.8.1

Parameters can depend on each other. For example, when testing a crossover of moving averages, it makes no sense to test a fast window that is longer than the slow window. By filtering out such cases, you only need to evaluate about half as many parameter combinations.

Test slow windows being longer than fast windows by at least 5

```
>>> @vbt.parameterized(merge_func="column_stack")
... def ma_crossover_signals(data, fast_window, slow_window):
...     fast_sma = data.run("sma", fast_window, short_name="fast_sma")
...     slow_sma = data.run("sma", slow_window, short_name="slow_sma")
...     entries = fast_sma.real_crossed_above(slow_sma.real)
...     exits = fast_sma.real_crossed_below(slow_sma.real)
...     return entries, exits

>>> entries, exits = ma_crossover_signals(
```

```
            (42, 48),
            (42, 49),
            (43, 48),
            (43, 49),
            (44, 49)],
           names=['fast_window', 'slow_window'], length=820)
```

Splitter

new in 1.8.0

Splitters in scikit-learn are not ideal for validating ML-based and rule-based trading strategies. VBT provides a juggernaut class that supports many splitting schemes that are safe for backtesting, including rolling windows, expanding windows, time-anchored windows, random windows for block bootstraps, and even Pandas-native `groupby` and `resample` instructions such as "M" for monthly frequency. As a bonus, the produced splits can be easily analyzed and visualized! For example, you can detect any split or set overlaps, convert all splits into a single boolean mask for custom analysis, group splits and sets, and analyze their distribution relative to each other. This class contains more lines of code than the entire backtesting.py package, so do not underestimate the new king in town! 

Tutorial

Learn more in the Cross-validation tutorial.

Roll a 360-day window and split it equally into train and test sets

```
>>> data = vbt.YFData.pull("BTC-USD", start="4 years ago")
>>> splitter = vbt.Splitter.from_rolling(
...     data.index,
...     length="360 days",
...     split=0.5,
...     set_labels=["train", "test"],
...     freq="daily"
... )
>>> splitter.plots().show()
```

Random search

new in 1.7.0

While grid search tests every possible combination of hyperparameters, random search selects and tests random combinations of hyperparameters. This is especially useful when there is a huge number of parameter combinations. Random search has also been shown to find equal or better

```
...     sl_stop=vbt.Param(stop_values),
...     tsl_stop=vbt.Param(stop_values),
...     tp_stop=vbt.Param(stop_values),
...     broadcast_kwargs=dict(random_subset=1000)   2
... )
>>> pf.total_return.sort_values(ascending=False)
sl_stop  tsl_stop  tp_stop
0.06     0.85      0.43       2.291260
         0.74      0.40       2.222212
         0.97      0.22       2.149849
0.40     0.10      0.23       2.082935
0.47     0.09      0.25       2.030105
                                 ...
0.51     0.36      0.01      -0.618805
0.53     0.37      0.01      -0.624761
0.35     0.60      0.02      -0.662992
0.29     0.13      0.02      -0.671376
0.46     0.72      0.02      -0.720024
Name: total_return, Length: 1000, dtype: float64
```

1. 100 combinations of each parameter = 100 ^ 3 = 1,000,000 combinations.
2. The indicator factory and parameterized decorator accept this argument directly.

Parameterized decorator

new in 1.7.0

There is a special decorator that allows any Python function to accept multiple parameter combinations, even if the function itself supports only one. The decorator wraps the function, gains access to its arguments, identifies all arguments acting as parameters, builds a grid from them, and calls the underlying function on each parameter combination from that grid. The execution can be easily parallelized. Once all outputs are ready, it merges them into a single object. Use cases are endless: from running indicators that cannot be wrapped with the indicator factory, to parameterizing entire pipelines! 🪄

Example 1: Basic SMA indicator

Parameterize a basic SMA indicator without Indicator Factory

```
>>> @vbt.parameterized(merge_func="column_stack")   1
```

Combination 30/30

```
window                              20            21            22  \
Date
2014-09-17 00:00:00+00:00          NaN           NaN           NaN
2014-09-18 00:00:00+00:00          NaN           NaN           NaN
2014-09-19 00:00:00+00:00          NaN           NaN           NaN
...                                ...           ...           ...
2024-03-07 00:00:00+00:00  57657.135156  57395.376488  57147.339134
2024-03-08 00:00:00+00:00  58488.990039  58163.942708  57891.045455
2024-03-09 00:00:00+00:00  59297.836523  58956.156064  58624.648793

...

window                              48            49
Date
2014-09-17 00:00:00+00:00          NaN           NaN
2014-09-18 00:00:00+00:00          NaN           NaN
2014-09-19 00:00:00+00:00          NaN           NaN
...                                ...           ...
2024-03-07 00:00:00+00:00  49928.186686  49758.599330
2024-03-08 00:00:00+00:00  50483.072266  50303.123565
2024-03-09 00:00:00+00:00  51040.440837  50846.672353

[3462 rows x 30 columns]
```

Example 2: Bollinger Bands pipeline

Parameterize an entire Bollinger Bands pipeline

```
>>> @vbt.parameterized(merge_func="concat")
... def bbands_sharpe(data, timeperiod=14, nbdevup=2, nbdevdn=2, thup=0.3, thdn=0.1):
...     bb = data.run(
...         "talib_bbands",
...         timeperiod=timeperiod,
...         nbdevup=nbdevup,
...         nbdevdn=nbdevdn
...     )
...     bandwidth = (bb.upperband - bb.lowerband) / bb.middleband
...     cond1 = data.low < bb.lowerband
...     cond2 = bandwidth > thup
...     cond3 = data.high > bb.upperband
...     cond4 = bandwidth < thdn
...     entries = (cond1 & cond2) | (cond3 & cond4)
...     exits = (cond1 & cond4) | (cond3 & cond2)
```

1. Use `concat` to merge metrics in the form of scalars and Series.
2. Builds the Cartesian product of 4 parameters.

Combination 16/16

```
nbdevup  nbdevdn  thup  thdn
1        1        0.4   0.1     1.681532
                        0.2     1.617400
                  0.5   0.1     1.424175
                        0.2     1.563520
         2        0.4   0.1     1.218554
                        0.2     1.520852
                  0.5   0.1     1.242523
                        0.2     1.317883
2        1        0.4   0.1     1.174562
                        0.2     1.469828
                  0.5   0.1     1.427940
                        0.2     1.460635
         2        0.4   0.1     1.000210
                        0.2     1.378108
                  0.5   0.1     1.196087
                        0.2     1.782502
dtype: float64
```

Riskfolio-Lib

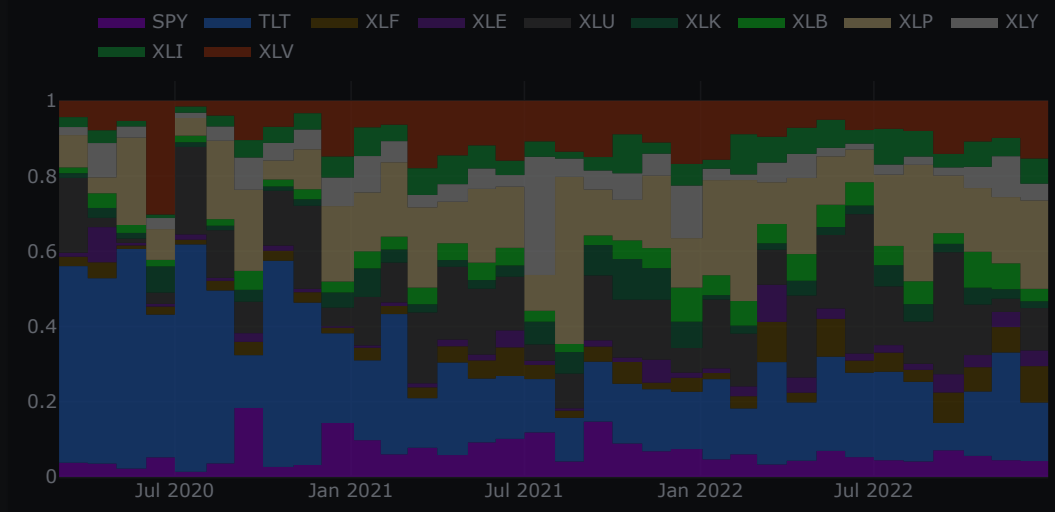
new in 1.7.0

Riskfolio-Lib is another increasingly popular library for portfolio optimization that has been integrated into VBT. Integration was done by automating typical workflows inside Riskfolio-Lib and putting them into a single function, so many portfolio optimization problems can be expressed using a single set of keyword arguments and easily parameterized.

Tutorial

Learn more in the Portfolio optimization tutorial.

```
...     port_cls="hc",
...     every="M"
... )
>>> pfo.plot().show()
```



Array-like parameters

new in 1.5.0

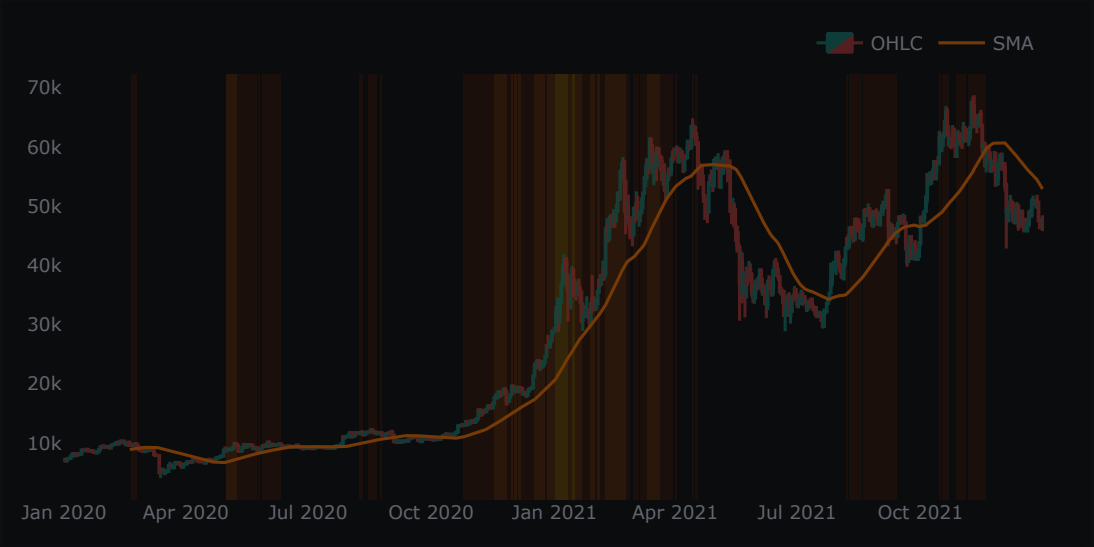
The broadcasting mechanism has been completely refactored and now supports parameters. Many parameters in VBT, such as SL and TP, are array-like and can be provided per row, per column, or even per element. Internally, even a scalar is treated as a regular time series and is broadcast along with other proper time series. Previously, to test multiple parameter combinations, you had to tile other time series so that all shapes matched perfectly. With this feature, the tiling procedure is performed automatically!

Write a steep slope indicator without indicator factory

```
>>> def steep_slope(close, up_th):
...     r = vbt.broadcast(dict(close=close, up_th=up_th))
...     return r["close"].pct_change() >= r["up_th"]
```

```
...         shape_kwargs=dict(fillcolor=color),
...         fig=fig
...     )
>>> plot_mask_ranges(0.005, "orangered")
>>> plot_mask_ranges(0.010, "orange")
>>> plot_mask_ranges(0.015, "yellow")
>>> fig.update_xaxes(showgrid=False)
>>> fig.update_yaxes(showgrid=False)
>>> fig.show()
```

1 Tests three parameters and generates a mask with three columns, one for each parameter.



Parameters

new in 1.5.0

There is a new module for working with parameters.

Generate 10,000 random parameter combinations for MACD

```
>>> from itertools import combinations

>>> window_space = np.arange(100)
>>> fastk_windows, slowk_windows = list(zip(*combinations(window_space, 2)))
```

```
...     build_index=False
... )
>>> pd.DataFrame(param_product)
      fast_window  slow_window  signal_window  macd_wtype  signal_wtype
0               0            1             47           3             3
1               0            2             21           2             2
2               0            2             33           1             1
3               0            2             42           1             1
4               0            3             52           1             1
...           ...          ...            ...         ...           ...
9995           97           99             19           1             1
9996           97           99             92           4             4
9997           98           99              2           2             2
9998           98           99             12           1             1
9999           98           99             81           2             2

[10000 rows x 5 columns]
```

1. Fast windows should be shorter than slow windows.
2. Fast and slow windows were already combined, so they share the same product level.
3. Window types do not need to be combined, so they share the same product level.

Portfolio optimization

new in 1.2.0

Portfolio optimization is the process of creating a portfolio of assets that aims to maximize return and minimize risk. Usually, this process is performed periodically and involves generating new weights to rebalance an existing portfolio. As with most things in VBT, the weight generation step is implemented as a callback by the user, while the optimizer calls that callback periodically. The final result is a collection of returned weight allocations that can be analyzed, visualized, and used in actual simulations 🍰

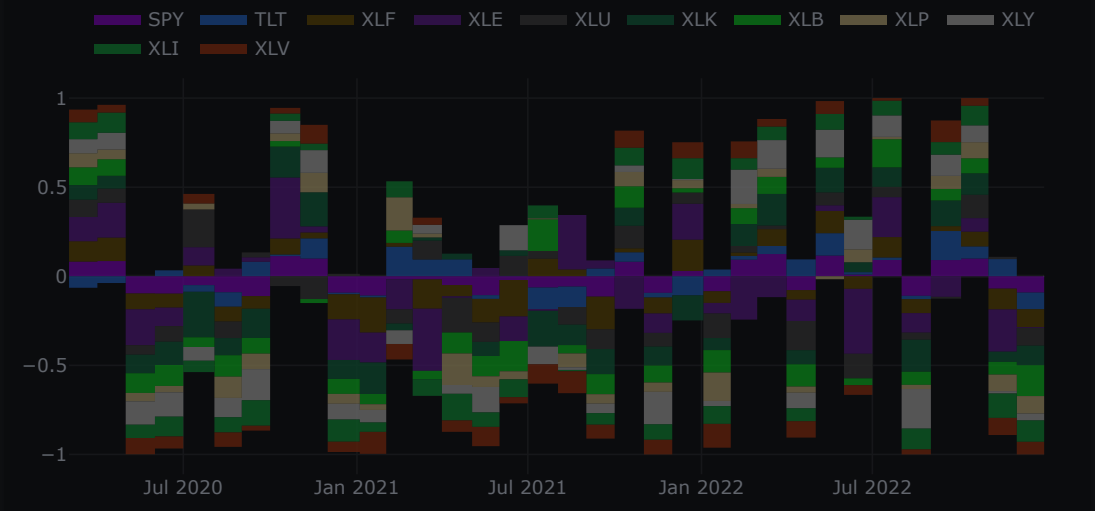
Tutorial

Learn more in the Portfolio optimization tutorial.

Allocate assets inversely to their total return in the last month

```
...         weights[neg_mask] = total_return[neg_mask] / total_return.abs().sum()
...     return -1 * weights

>>> data = vbt.YFData.pull(
...     ["SPY", "TLT", "XLF", "XLE", "XLU", "XLK", "XLB", "XLP", "XLY", "XLI", "XLV"],
...     start="2020",
...     end="2023",
...     missing_index="drop"
... )
>>> pfo = vbt.PFO.from_optimize_func(
...     data.symbol_wrapper,
...     regime_change_optimize_func,
...     vbt.RepEval("data[index_slice]", context=dict(data=data)),
...     every="M"
... )
>>> pfo.plot().show()
```

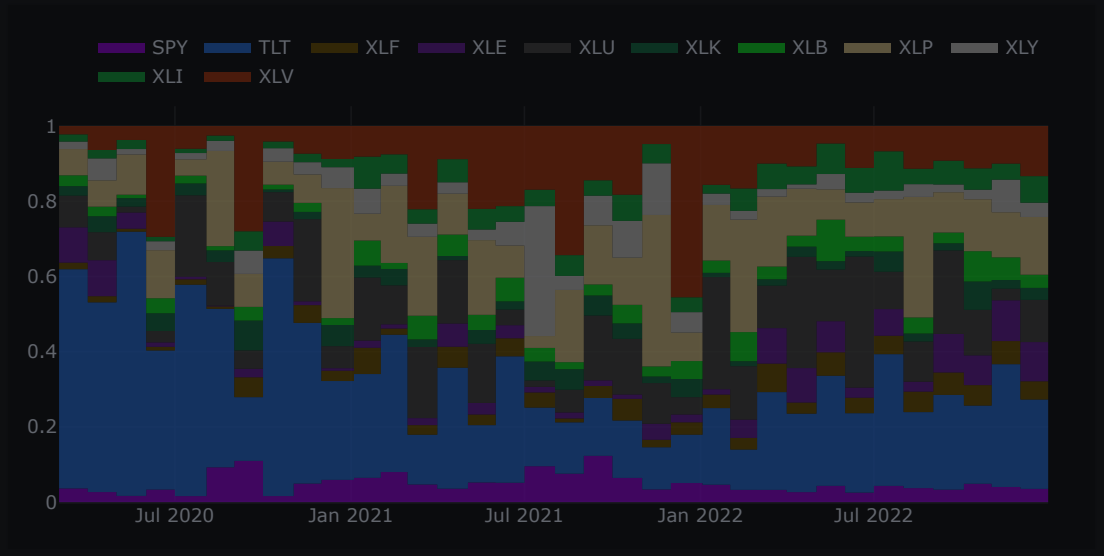


PyPortfolioOpt

new in 1.2.0

PyPortfolioOpt is a popular financial portfolio optimization package that includes both classical methods (Markowitz 1952 and Black-Litterman), suggested best practices (such as covariance shrinkage), and many recent developments and novel features, like L2 regularization, shrunk

```
>>> data = vbt.YFData.pull(
...     ["SPY", "TLT", "XLF", "XLE", "XLU", "XLK", "XLB", "XLP", "XLY", "XLI", "XLV"],
...     start="2020",
...     end="2023",
...     missing_index="drop"
... )
>>> pfo = vbt.PFO.from_pypfopt(
...     returns=data.returns,
...     optimizer="hrp",
...     target="optimize",
...     every="M"
... )
>>> pfo.plot().show()
```



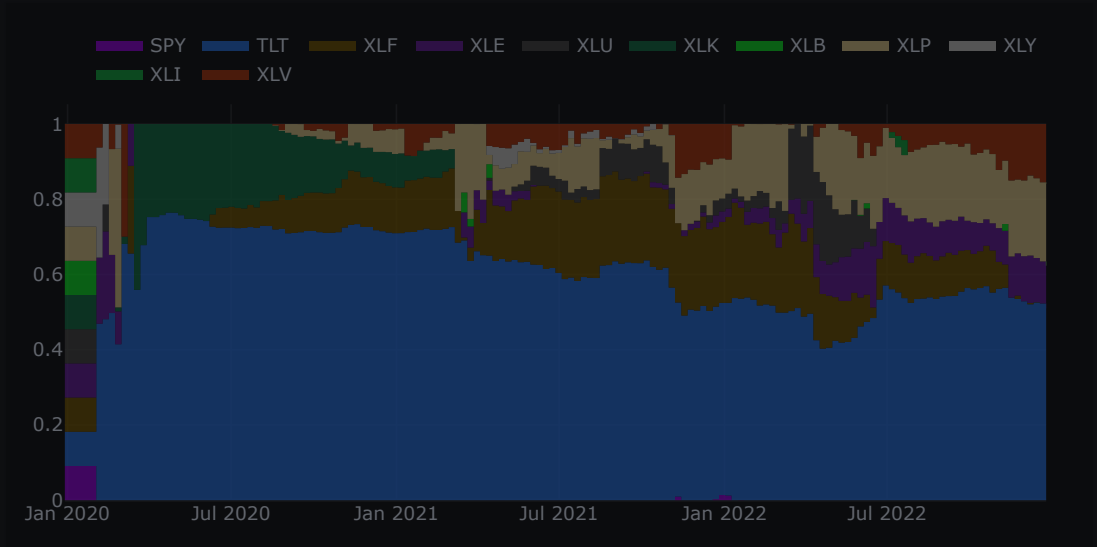
Universal Portfolios

new in 1.2.0

Universal Portfolios is a package that brings together various Online Portfolio Selection (OLPS) algorithms.

Tutorial

```
...     missing_index="drop"
... )
>>> pfo = vbt.PFO.from_universal_algo(
...     "MPT",
...     data.resample("W").close,
...     window=52,
...     min_history=4,
...     mu_estimator='historical',
...     cov_estimator='empirical',
...     method='mpt',
...     q=0
... )
>>> pfo.plot().show()
```



And many more...

- Look forward to more killer features being added every week!